

9. Gün Notlarım: Projeyi Paketlemek ve Otomatik Test Ettirmek

Kod artık çalışıyordu ama aklımda şu soru vardı: "Bu proje benim bilgisayarında çalışıyor, ama başka birinin bilgisayarında da aynı şekilde çalışacağını nasıl garanti ederim?" Bugün bunun cevabını Docker ve otomatik test/kontrol sistemleriyle öğrendim.

Docker neden gerekli?

Benim bilgisayarında Python'un belirli bir sürümü, belirli kütüphaneler kurulu. Ama başka birinin bilgisayarında bunlar farklı olabilir, hatta hiç olmayabilir. Docker bana şunu sağlıyor: "içinde her şeyin hazır olduğu, kendi kendine yetebilen bir kutu" oluşturmak. Bu kutuya imaj (image) deniyor, çalışan haline de **konteyner** (container) deniyor.

Ben iki ayrı kutu hazırladım:

- Biri backend için — Python ve gerekli kütüphaneler içinde hazır kurulu.
- Biri de frontend için — nginx denen bir web sunucusu, sadece dosyaları servis

ediyor ve gelen istekleri backend'e yönlendiriyor (buna "proxy" deniyor).

Testleri de kutunun içinde çalıştırmak

Öğrendiğim ilginç bir teknik: aynı Docker dosyası içinde birden fazla "katman" tanımlanabiliyor. Ben normal çalışan uygulamanın yanında ayrı bir "test katmanı" da tanımladım. Bu sayede "kodu test ederken kullandığım ortam" ile "gerçekte çalışan ortam" birbirinden çok farklı olmuyor — testler de aynı kutunun içinde, aynı koşullarda çalışıyor.

İki servisin birbirini beklemesi

docker-compose ile backend ve frontend'i aynı anda ayağa kaldırıyorum. Ama bir sorun fark ettim: frontend, backend henüz hazır olmadan istek atmaya kalkarsa hata alıyor. Bunu çözmek için "healthcheck" denen bir mekanizma öğrendim — backend'e düzenli aralıklarla "sen hazır mısın?" diye soran bir kontrol var, ve frontend ancak backend bu soruya "evet hazırım" diyene kadar beklemeye ayarlandı.

GitHub Actions ile testlerin otomatik çalışması

7. günde GitHub Actions'ın temel mantığını (workflow, job, step) öğrenmiştim. Bugün bunu gerçek bir kullanım senaryosunda uyguladım: kod her push edildiğinde ya da bir pull request açıldığında, GitHub kendiliğinden şu adımları sırayla çalıştırıyor:

1. Önce backend'deki kod stiline uyulup uyulmadığını kontrol ediyor ve testleri

çalıştırıyor.

2. Sonra frontend tarafındaki dosyaların bozuk olup olmadığına bakıyor.

3. Ardından her iki Docker imajını gerçekten inşa edip (build) çalıştırıp çalışmadığını doğruluyor.

4. En son da tüm sistemi (`docker compose up` ile) gerçekten ayağa kaldırıp, gerçek bir isteğin (örneğin bir hız komutunun) beklenen sonucu üretip üretmediğini kontrol ediyor.

Bunun bana kattığı şey şu oldu: artık her değişiklik yaptığımda "acaba bir şeyi bozdum mu" diye elle kontrol etmek zorunda kalmıyorum, bu dört adım benim yerime kontrol ediyor ve bir şey bozulursa bana anında haber veriyor.

Ayrıca workflow dosyasındaki GitHub Actions sürümlerini de güncel tuttum, çünkü eski sürümler zamanla kullanımdan kaldırılıyor (deprecated) ve uyarı veriyor.

Testleri neden üçe böldüm

Testlerimi tek bir dosyada değil, konularına göre üç ayrı dosyada topladım:

- Biri sadece mesajın doğru kodlanıp çözüldüğünü kontrol ediyor (matematik doğru mu).
- Biri simülatörün davranışını kontrol ediyor (hız gerçekten yumuşak artıyor mu,

fren gerçekten yavaşlatıyor mu).

- Biri de API'nin dışarıdan gelen isteklere doğru cevap verip vermediğini kontrol ediyor.

Bu ayrım sayesinde bir test kırıldığında hangi katmanda sorun olduğunu isim üzerinden hemen anlayabiliyorum.

Ayarları kod içine gömmemek

Son olarak öğrendiğim şey: hız sınırı, güncelleme hızı gibi değerleri kodun içine sabit olarak yazmak yerine ortam değişkeni (environment variable) üzerinden okunabilir hale getirdim. Böylece kodu hiç değiştirmeden, sadece ayar dosyasından (`.env`) bu değerleri değiştirebiliyorum — mesela test ortamında daha düşük bir hız sınırı kullanmak istersem kod dokunmadan yapabiliyorum.